

— SYSTEM DESIGN DOCUMENT

WhatsApp Lead Agent — a white-label conversational CRM

An inbound WhatsApp agent that qualifies and scores leads automatically, a manual/CSV intake path for leads sourced elsewhere, and an internal dashboard sales teams use to work them — built so any CJK client can run it under their own number and brand.

Prepared by Gagan Hari Kiran **Prepared for** CJK Technologies **Version** 1.0 **Date** 16 Sep 2026
Status Draft for review

Inbound qualification & scoring

PostgreSQL persistence

Manual + CSV lead intake

Dashboard, filters, CSV export

Scheduled follow-ups

Qualified-lead email alerts

Hindi + Indian languages

Multi-tenant / white-label

CONTENTS

- 01 Overview & goals
- 02 Architecture
- 03 Multi-tenancy & white-label
- 04 Conversation & qualification
- 05 Lead scoring engine
- 06 Manual & CSV ingestion
- 07 Outbound & follow-ups
- 08 Internal dashboard
- 09 Email alerts

- 10 Language support
- 11 Data model
- 12 Security & reliability
- 13 Stack & rollout

Goals, scope, and what "done" means for v1

One system, two ways in for a lead — it starts a WhatsApp conversation, or a salesperson puts it there — and one place sales works from afterward.

The product sits between a client's WhatsApp Business number and their sales team. A prospect messages in, the agent runs a qualification conversation and scores the lead as it goes; a salesperson can also drop in leads collected elsewhere (a form, an event, a purchased list) as a CSV or one at a time. Every lead, every message, and every state change is stored in PostgreSQL and worked from one internal dashboard, and the system is built multi-tenant from day one so each CJK client runs it under their own WhatsApp number, branding, and language mix without a fork of the codebase.

In scope for v1

- **Inbound conversation, qualification, and scoring** — a scripted-but-conversational flow that asks qualifying questions, extracts answers, and produces a lead score in real time.
- **Full persistence** — every lead, contact, transcript, and conversation state lives in PostgreSQL; nothing important lives only in a queue or a log file.
- **Internal dashboard** — list, filter, and search leads; export the filtered set to CSV; view a lead's full transcript and score history.
- **Manual and CSV lead intake** — add a lead by hand, or bulk-import a CSV with column mapping, validation, and de-duplication.
- **Outbound template messages and scheduled follow-ups** — campaigns and time- or state-triggered follow-up sequences, respecting Meta's messaging rules.
- **Qualified-lead email alerts** — the moment a lead crosses the qualified threshold, sales gets an email with the lead snapshot and a link to the transcript.
- **Indian-language support** — Hindi and English at minimum, with the pipeline designed to add Tamil, Telugu, Kannada, Bengali, Marathi, Gujarati, Malayalam, Punjabi, and Odia without architectural change.

Explicitly out of scope for v1

- Payments, invoicing, or checkout inside the chat (WhatsApp Commerce/catalog is a later add-on, not v1).
- Other channels — SMS, Instagram DM, voice. The agent is WhatsApp-only for now; the schema leaves room for a channel column, but nothing else is built.

- Self-serve tenant onboarding (a client connecting their own WhatsApp number without CJK's involvement) — v1 onboarding is done by CJK for each client; see §13.
- Analytics beyond list/filter/export — funnel dashboards, attribution, cohort reporting are phase-3 candidates.

Non-functional requirements

REQUIREMENT	TARGET FOR V1
Tenant isolation	Hard isolation enforced in the database (row-level security), not only in application code — a bug in one tenant's API layer cannot leak another tenant's leads.
Latency	Webhook acknowledged to Meta in <5s (Meta's own requirement); conversational reply sent within ~3s p95 of the inbound message being enqueued.
Availability	99.5% for v1, single-region. No multi-region failover in v1 — noted as a phase-3 item if a client's SLA needs it.
Auditability	Full transcript and score-history retention; every manual override (status, score, ownership) is written to an immutable audit log with actor and reason.
Extensibility	Onboarding a new tenant is a configuration and data operation (WABA credentials, branding, qualification script, scoring weights) — never a code change or a deploy.
Compliance	Meta's WhatsApp Business Messaging Policy (opt-in, 24-hour session window, template categories) and India's Digital Personal Data Protection Act, 2023, for any tenant handling Indian consumer data.

— 02 · ARCHITECTURE

Component architecture

One queue and one database sit at the centre; everything else is a worker that reads from or writes to them.

Inbound messages arrive as webhooks from the WhatsApp Business Platform, get acknowledged immediately, and are handed to a queue so the slow work — language detection, scoring, LLM calls, persistence — happens asynchronously and Meta never sees a timeout. Outbound traffic (replies, scheduled follow-ups, campaigns) goes back out through the same queue so both directions share retry, rate-limiting, and logging logic. The dashboard, the CSV importer, and the notification service all read and write the same PostgreSQL database rather than talking to each other directly — the database, not a service mesh, is the integration point.

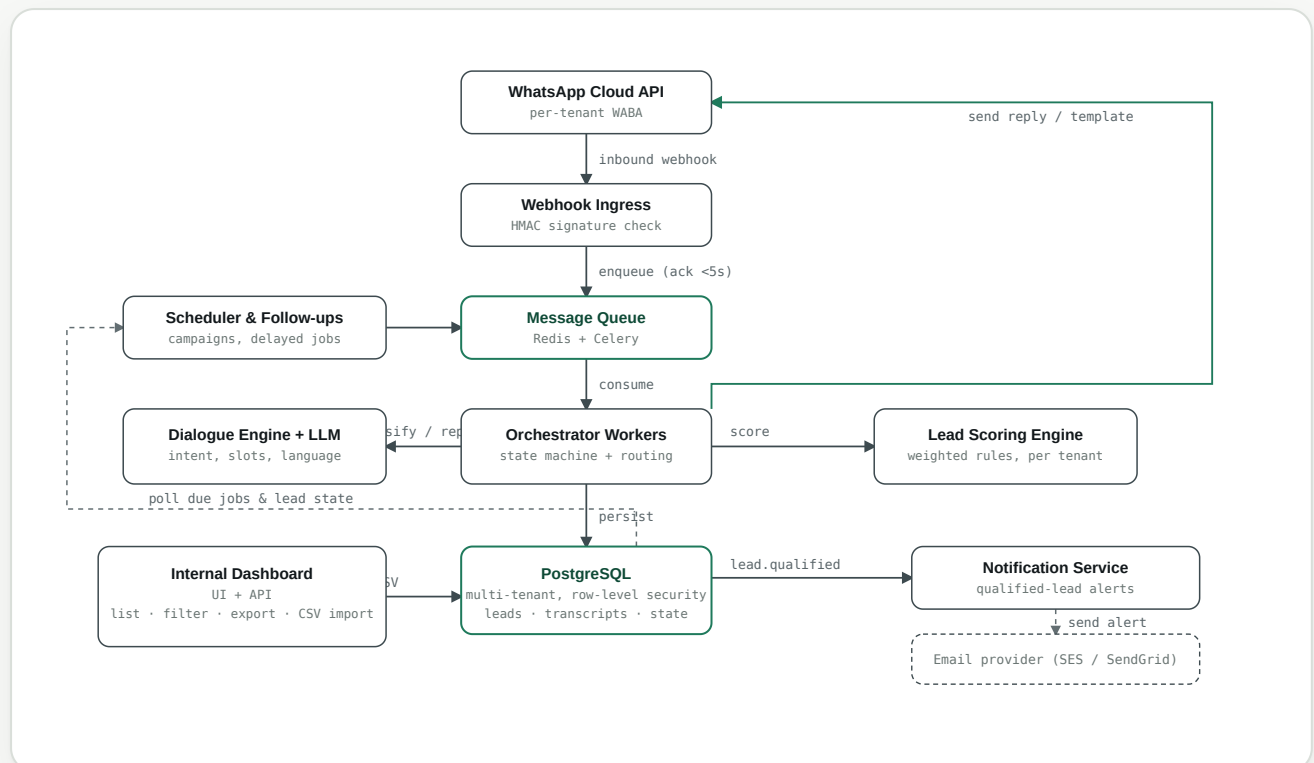


Fig. 1 — Two boxes carry the whole system: the **Message Queue** decouples every inbound and outbound send from real-time constraints, and **PostgreSQL** is the single source of truth every other component reads from or writes to — there is no service-to-service API between the dashboard, the scheduler, and the notification service.

Component responsibilities

COMPONENT	RESPONSIBILITY
Webhook Ingress	Verifies the X-Hub-Signature-256 HMAC on every inbound Meta webhook, resolves the tenant from phone_number_id, and enqueues the raw event within Meta's 5-second ack window. Does no business logic.
Message Queue	Redis-backed job queue (Celery) carrying both inbound events and outbound sends. Gives retries, backoff, per-tenant rate limiting, and dead-letter handling for free.
Orchestrator Workers	Horizontally-scaled consumers that run the conversation state machine, decide the next action, and call the two services below. Idempotent per wa_message_id.
Dialogue Engine + LLM	Language detection, intent/slot extraction against the tenant's qualification script, and free-text fallback replies. Deterministic script drives the critical path; the LLM handles the parts language actually varies.
Lead Scoring Engine	Applies the tenant's weighted scoring config to the conversation so far and returns a score + human-readable reasons, written to lead_score_history.
Scheduler & Follow-ups	Polls for due follow-up jobs and campaign sends, respects per-tenant WhatsApp messaging-tier caps, and enqueues outbound sends onto the same Message Queue.
Notification Service	Subscribes to the lead.qualified event, renders a tenant-branded email, and sends via the transactional email provider with retry and delivery logging.
Internal Dashboard	The sales-facing UI and its API: lead list/filter/export, transcript viewer, CSV import wizard, campaign and template management, tenant branding.

◆ BUILD VS. BUY – THE WHATSAPP LAYER ITSELF

Two credible paths to the "WhatsApp Cloud API" box: go direct with Meta as a Tech Provider (full control, lowest marginal cost, but CJK owns template submission, number quality, and WABA management for every client), or resell through an Indian BSP with a white-label/partner program — AiSensy, Interakt, Wati, and Gupshup all support this and are built for exactly this reseller pattern.

Recommendation: start pilot tenants on a BSP reseller plan — it gets a client live in days, not weeks, and the BSP handles template approval UX and number quality scoring. Put a thin WhatsAppGateway interface between the Orchestrator and whichever provider is behind it, so moving a tenant to direct Cloud API later (once volume justifies the integration work) is a configuration change, not a rewrite.

— 03 · MULTI-TENANCY

White-label & multi-tenancy model

One deployment, many clients — each invisible to the others, each looking like their own product.

Default posture is a **shared database, shared schema** model: every tenant-scoped table carries a `tenant_id` column, and PostgreSQL row-level security policies enforce isolation at the database layer — an application bug that forgets a `WHERE tenant_id = ...` clause still cannot return another tenant's rows. This is the cheapest model to operate and onboard new clients into. A schema-per-tenant (or database-per-tenant) escape hatch stays available for any client whose contract demands stronger physical isolation; it costs more to operate and should be the exception, not the default.

What's white-labelable per tenant

SURFACE	CONFIGURABLE PER TENANT
WhatsApp identity	Own WABA, own display number and business name, own message templates and approval history.
Dashboard	Logo, accent colour, sender display name; served at a tenant subdomain (<code>{tenant}.leads.cjktechnologies.in</code>) or the client's own CNAME.
Email alerts	Sender name/address, email template branding, recipient distribution list.
Qualification script	The questions asked, their order, and which answers map to which scoring signals.
Scoring model	Signal weights and the score thresholds that define New / Contacted / Qualified / Unqualified.
Languages	Which of the supported locales are active, and the default fallback language.
Follow-up cadence	Delay rules and templates used for scheduled re-engagement.

CJK itself is a reserved `super_admin` tenant with cross-tenant visibility for support and operations — this is the only role that ever queries across tenant boundaries, and every such query is logged to the audit trail.

— 04 • CONVERSATION

Conversation & qualification flow

A deterministic script drives the questions asked and the score they produce; the LLM only handles the parts that are genuinely open-ended — free text, and which language it's in.

Lead scoring feeds a CRM that a compliance-minded sales team will eventually ask "why is this lead scored 78?" — so the qualification path is a state machine with an explicit script, not an LLM improvising a sales conversation. The LLM is scoped to

three jobs: detecting the customer's language, extracting a structured answer out of a free-text reply, and handling the fallback case where the customer says something the script didn't anticipate.

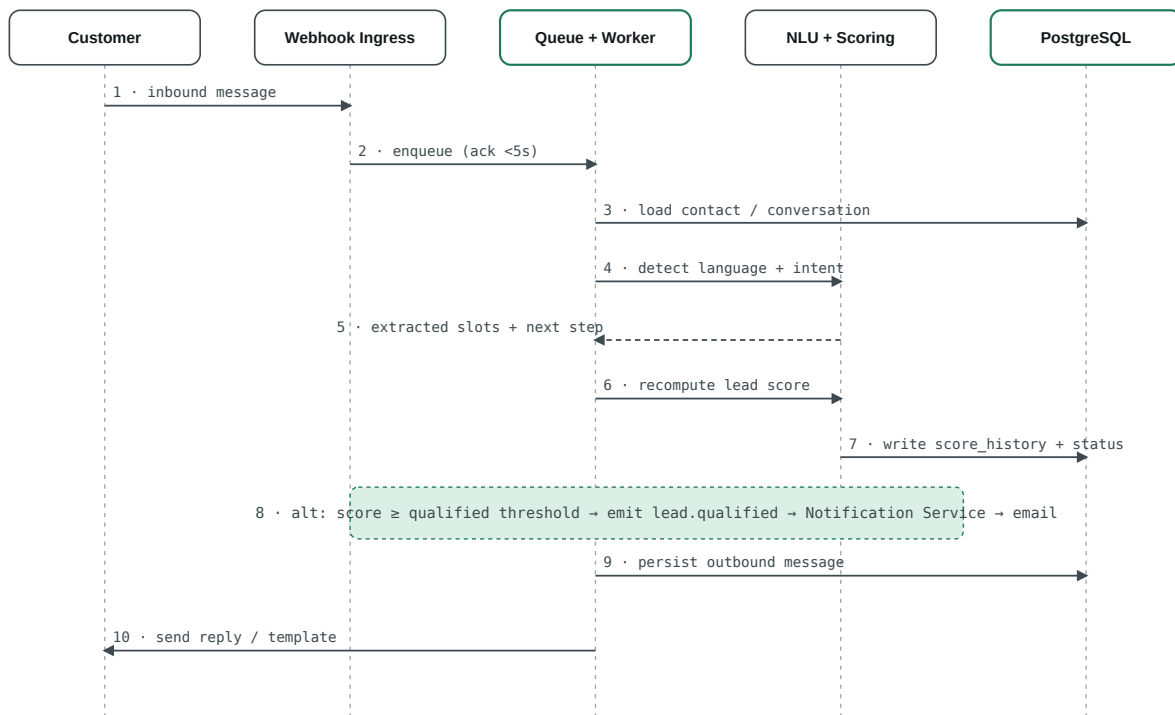


Fig. 2 — The qualified-lead email (\$9) is not a separate poller — it fires inline, step 8, the instant a score crossing is written, so there is no lag between "lead qualifies" and "sales is notified."

Conversation state machine

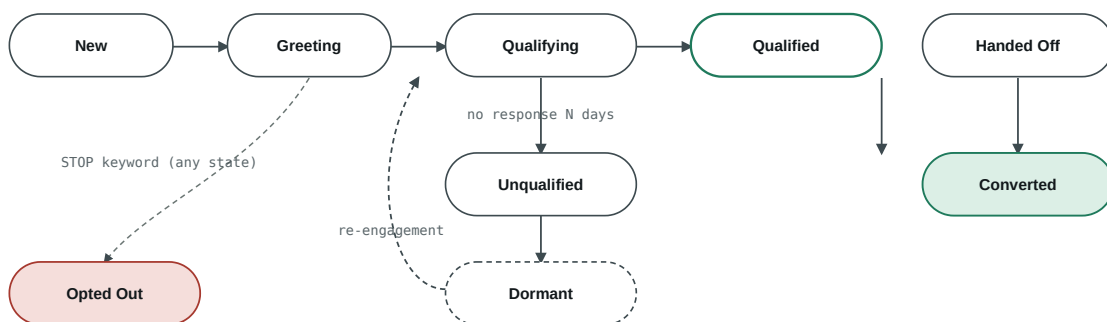


Fig. 3 — Opted Out is reachable from every state on a STOP keyword or Meta-managed opt-out — drawn once here for clarity rather than as nine separate arrows.

META'S 24-HOUR WINDOW, IN PRACTICE

Once a customer messages in, free-form replies are allowed for 24 hours from their last message; outside that window, only a pre-approved template message can re-open the conversation. The Orchestrator checks `last_customer_message_at` before every send and automatically substitutes the tenant's approved template when the session window has lapsed — sales and the qualification script never need to think about this.

— 05 • SCORING

Lead scoring engine

A weighted, per-tenant rule set — transparent enough that a salesperson can see exactly why a lead scored what it did.

Each signal below contributes points on a 0–100 scale; the config is stored as JSON on the tenant record so a client can tune it without a deploy. A worked example:

```
{
  "signals": {
    "source_quality":      { "weight": 15, "map": { "referral": 15, "inbound_organic": 10 },
    "stated_budget":       { "weight": 25, "map": { "confirmed": 25, "implied": 12,
    "timeline_urgency":    { "weight": 20, "map": { "immediate": 20, "this_quarter": 10 },
    "questions_completed": { "weight": 20, "per_question": 4, "max_questions": 5 },
    "response_latency":    { "weight": 10, "under_1h": 10, "under_24h": 5, "over_24h": 0 },
    "engagement_recency":  { "weight": 10, "decay_per_day": 1.5 }
  },
  "thresholds": { "unqualified_below": 30, "contacted_below": 55, "qualified_at": 70 }
}
```

SCORE BAND	STATUS	MEANING
0–29	• Unqualified	Insufficient signal or explicit disqualification (wrong geography, no budget, not a decision-maker).
30–54	• New / Contacted	Engaged but qualification incomplete.
55–69	• Qualifying	Meaningful signal, still short of the bar.
70–100	• Qualified	Crosses the tenant's threshold → triggers <code>lead.qualified</code> exactly once (idempotent on <code>qualified_at</code>) → email alert (\$9).

Every recompute is appended to `lead_score_history` with the signal breakdown that produced it, not just the final number — the "why" is what a sales manager will ask for first.

— 06 • MANUAL & CSV INTAKE

Manual and CSV lead ingestion

Leads don't only arrive by WhatsApp — a salesperson can add one by hand or drop in a spreadsheet from an event or a purchased list.

Manual entry & bulk selection

A dashboard form creates a contact + lead directly, in whatever status the rep chooses. The lead list also supports multi-select for bulk actions — assign an owner, add a tag, enroll in a campaign, or export — so a rep working a spreadsheet-derived shortlist never has to touch each row individually.

CSV import pipeline

- 1 **Upload & map** — the rep uploads a CSV and maps its columns to fields (name, phone, source, tags, custom notes); the mapping is remembered per tenant for next time.
- 2 **Validate** — every phone number is normalized to E.164 (via the `phonenumbers` library — Python's port of Google's `libphonenumber` — using the tenant's default

region as fallback for numbers without a country code); rows that don't validate are flagged, not silently dropped.

- 3 **De-duplicate** — matched against existing contacts by (`tenant_id`, `phone_e164`); the tenant configures whether a match is skipped, merged, or opened as a new conversation thread.
- 4 **Process asynchronously** — large files are handed to the same worker pool as everything else, so a 20,000-row import doesn't block the UI or starve inbound message processing.
- 5 **Report** — a downloadable per-row result (imported / skipped-duplicate / failed-with-reason) so the rep can fix and re-upload just the failures.
- 6 **Optional enrollment** — successfully imported leads can be enrolled straight into an outbound template campaign (§7) as the last step of the wizard.

IMPORT STATE	MEANING
uploaded	File received, queued for validation.
validating	Column mapping applied, phone numbers normalized, duplicates flagged.
processing	Valid rows being written as contacts/leads.
completed	All rows processed without errors.
completed_with_errors	Processed, but some rows were skipped — report available.
failed	File-level failure (bad encoding, missing required column) — nothing written.

— 07 • OUTBOUND

Outbound templates & scheduled follow-ups

Broadcasts and drip sequences share the same queue, the same rate limiter, and the same opt-out check as every other outbound send.

Every tenant maintains a catalog of Meta-approved message templates per language. A campaign is either an immediate broadcast to a filtered lead segment, a scheduled

broadcast, or a follow-up sequence triggered by lead status or time-since-last-contact. A few representative follow-up rules:

TRIGGER	DELAY	ACTION
Lead reaches New , no reply	+4 hours	Send "still there?" nudge template
Qualifying started, question left unanswered	+24 hours	Send reminder template with the same question rephrased
Lead reaches Qualified , no rep contact logged	+2 hours	Escalation email to sales manager (phase 2)
No activity for 14 days	—	Move to Dormant; one re-engagement template, then stop

RATE LIMITS ARE PER TENANT, NOT SHARED

Because each tenant has its own WABA, Meta's messaging-tier caps (starting at 250 unique customers per rolling 24 hours, scaling up with number quality) apply per tenant — one client's campaign volume can never throttle another's. The Scheduler still needs to track each tenant's current tier and pace sends under it, and to immediately halt all scheduled jobs for a contact the moment they reply STOP or opt out through Meta's own mechanism.

— 08 • DASHBOARD

Internal dashboard

Built for a sales rep working a list, not for an analyst — filters and bulk actions come before charts.

Lead list — illustrative view

LEAD	PHONE	SOURCE	SCORE	STATUS
Rina Deshmukh · Urban Nest Interiors	+91 98200 11234	WhatsApp inbound	82	• Qualified
Sameer Anand · Anand Traders	+91 90040 55621	CSV import	46	• Contacted
Farah Sheikh	+91 88888 30192	WhatsApp inbound	61	• Qualifying
GoodFunds Capital · S. Iyer	+91 98765 44120	Manual entry	91	• Qualified
Deepak Mehra	+91 77021 98342	WhatsApp inbound	18	• Unqualified

Sample rows shown for illustration. Filters: status, score range, source, date range, owner, tag, language. Bulk actions on any selection: assign owner, add tag, enroll in campaign, export CSV.

Other screens

- **Lead detail** — full transcript with per-message delivery ticks, score breakdown, a timeline of every state change, and a manual override control (status/score/owner) that always asks for a reason and writes it to the audit log.
- **Campaigns & templates** — create/schedule campaigns, and track each template's Meta approval status per language.
- **Branding** — tenant admin controls logo, colour, sender identity (super_admin and tenant_admin only).
- **Users & roles** — manage who on the client's team can see and do what.

API surface (representative)

METHOD & PATH	PURPOSE
GET /leads	List with filters, pagination, sort — backs the lead list.
GET /leads/:id	Lead detail, including score breakdown and owner.
PATCH /leads/:id	Manual override of status/owner/score (requires reason; audited).
GET /leads/export	Streams the current filtered set as CSV.
GET /conversations/:id/messages	Full transcript for the detail view.
POST /csv-imports	Upload + column mapping; returns an import id to poll.
GET /csv-imports/:id	Import status and per-row report.
POST /campaigns	Create/schedule an outbound campaign against a segment.
GET /templates	Template catalog and per-language approval status.
GET /tenants/:id/settings	Branding, scoring config, active languages (tenant_admin+).

Roles

ROLE	CAN DO
super_admin (CJK)	Everything, across all tenants; every cross-tenant read is audit-logged.
tenant_admin	Everything within their tenant, including branding, scoring config, and user management.
sales_rep	List/filter/export leads, work transcripts, override status/owner on leads assigned to them.
viewer	Read-only — list, filter, export, view transcripts. No overrides.

— 09 • ALERTS

Qualified-lead email alerts

One event, one email, logged either way.

The `lead.qualified` domain event (\$5) triggers the Notification Service to render a tenant-branded email — lead snapshot, score breakdown, and a direct link into the dashboard transcript — and send it via a transactional provider. **Amazon SES** is the default recommendation (cost at scale, an India region for data residency); SendGrid or Postmark are drop-in alternatives if a client's contract prefers them. Every send attempt, success or failure, is written to a notifications table with retry/backoff on failure, and the trigger is idempotent — a lead that drops out of Qualified and re-qualifies later sends exactly one new alert, not a flood of re-alerts on every score recompute in between.

— 10 • LANGUAGE

Multi-language support

Hindi and English at launch; the pipeline is built for the rest of the list from day one, not bolted on later.

PHASE	LANGUAGES
v1 (must-have)	Hindi, English — plus common Romanized "Hinglish" code-mixed text
Phase 2 (tenant-configurable)	Tamil, Telugu, Kannada, Bengali, Marathi, Gujarati, Malayalam, Punjabi, Odia

Pipeline

- 1 Detect** — script-based detection is near-certain for Devanagari, Tamil, Telugu, etc.; Romanized/code-mixed text needs a language-ID classifier trained on that pattern (script heuristics alone under-perform on Hinglish).
- 2 Respond in the script's own language** — canned qualification-script replies are stored per locale; the LLM fallback for free text is instructed to reply in the detected language, not translate afterward.
- 3 Translate campaign copy** — a pluggable Translator interface (Google Cloud Translation, Azure Translator, or Bhashini) drafts a first-pass translation of tenant-authored campaign copy into every active language; a human reviews it before submission, because Meta requires a separately approved template per language variant, not just per campaign.
- 4 Let sales read anything** — the dashboard transcript view offers an on-demand translation of any message into the viewing rep's preferred language, which matters for CJK's own ops staff supporting a Tamil-speaking tenant's conversations, for example.

— 11 • DATA MODEL

Data model

Everything the conversation touches is a row before it's anything else.

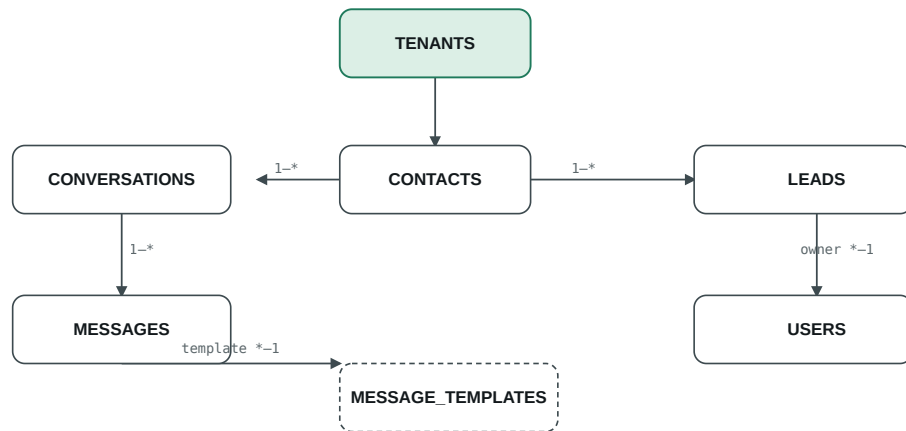


Fig. 4 — Core lifecycle only. Every table below also carries `tenant_id` and is subject to the same row-level security policy as `CONTACTS`, whether or not an arrow is drawn to `TENANTS` here.

Schema reference

TABLE	KEY COLUMNS	PURPOSE
tenants	id, slug, brand_config jsonb, plan, status	One row per CJK client.
waba_accounts	tenant_id, phone_number_id, waba_id, access_token_ref, default_locale	Per-tenant WhatsApp credentials; token stored by reference to a secrets manager, never in plaintext.
users	tenant_id, email, role, status	Dashboard logins; tenant_id null only for super_admin.
contacts	tenant_id, phone_e164, name, locale, opted_in, opted_in_at	One row per WhatsApp number per tenant.
leads	tenant_id, contact_id, owner_user_id, status, score, source, qualified_at	The sales-facing record; a contact may have more than one lead over time.
lead_score_history	lead_id, score, breakdown jsonb, created_at	Append-only trail of every score recompute and why.
conversations	tenant_id, contact_id, lead_id, state, current_flow_step, locale	One active WhatsApp thread; carries the state-machine position.
messages	conversation_id, direction, wa_message_id (unique), type, content jsonb, status	Full transcript; unique wa_message_id makes webhook redelivery idempotent.
message_templates	tenant_id, locale, meta_template_id, category, status	Meta-approved templates, one row per language variant.
campaigns	tenant_id, template_id, audience_filter jsonb, schedule_type, status	Broadcasts and drip sequences.
follow_up_rules	tenant_id, trigger_status, delay_minutes, action	Declarative re-engagement rules (§7 table).
scheduled_jobs	tenant_id, job_type, payload jsonb, run_at, status, attempts	Queue-backed durable jobs for follow-ups and campaigns.

TABLE	KEY COLUMNS	PURPOSE
csv_imports	tenant_id, uploaded_by, mapping jsonb, success_count, error_count, status	One row per import batch (\$6).
notifications	tenant_id, lead_id, type, recipient, status, sent_at	Every alert send attempt, for audit and de-duplication.
tags / lead_tags	tenant_id, name / lead_id, tag_id	Free-form labeling used by dashboard filters.
audit_log	tenant_id, actor_user_id, action, entity, entity_id, metadata jsonb	Immutable record of every manual override and cross-tenant read.

Tenant isolation, mechanically

```

ALTER TABLE leads ENABLE ROW LEVEL SECURITY;

CREATE POLICY tenant_isolation ON leads
  USING (tenant_id = current_setting('app.tenant_id')::uuid);

-- every request sets this once, right after acquiring the DB connection:
-- SELECT set_config('app.tenant_id', $1, false);

```

Applied identically to every tenant-scoped table. The application still filters by tenant in its queries for clarity and index usage — RLS is the backstop, not the only line of defense.

Security, compliance & reliability

CONCERN	APPROACH
Webhook authenticity	HMAC verification of X-Hub-Signature-256 on every inbound Meta webhook; requests that fail are rejected before touching the queue.
Idempotency	Unique constraint on messages.wa_message_id — Meta's at-least-once webhook delivery can redeliver safely.
Secrets	WABA access tokens and provider API keys held in a secrets manager, referenced (not stored) from PostgreSQL.
Data residency	Primary database and object storage hosted in an India region (e.g. AWS ap-south-1 / Mumbai) as the default for India-DPDP-Act-relevant tenants.
Consent	WhatsApp's own opt-in is treated as the consent record (opted_in_at); a STOP keyword or Meta-managed opt-out immediately halts all outbound to that contact.
Retention	Transcript and PII retention period configurable per tenant contract; deletion is a hard delete, not a soft flag, once the retention window lapses.
Backups	Automated PostgreSQL snapshots with point-in-time recovery; target RPO 1 hour / RTO 4 hours for v1.
Scaling	Orchestrator workers scale horizontally behind the queue; the queue and per-tenant WABA rate limits are the natural backpressure points.

Recommended stack & phased rollout

Stack

LAYER	CHOICE	WHY
Backend	Python 3.12 + FastAPI	Async request handling suits the I/O-bound webhook workload; Pydantic models double as request/response validation and the scoring-config schema, and FastAPI's Depends () injection maps cleanly onto per-tenant guards in a modular monolith that can be split later.
Queue	Redis + Celery	Celery Beat's cron-like scheduling covers delayed follow-ups and campaign sends alongside real-time task routing, on the same Python-native stack as the API — no second language to operate.
Database	PostgreSQL 15+, managed (RDS/Supabase), ap-south-1	Row-level security for tenant isolation; mature, boring, well understood by any future CJK hire.
Object storage	S3-compatible	CSV files, exports, media attachments.
Frontend	React + TypeScript + Tailwind	Standard SPA dashboard, JWT-authenticated, tenant-scoped.
LLM / NLU	Claude via API	Language detection, slot extraction, and free-text fallback replies across Indian languages.
WhatsApp layer	BSP reseller for pilots → direct Meta Cloud API at scale	See the build-vs-buy callout in §2.
Email	Amazon SES (SendGrid/Postmark as alternatives)	Cost at scale, India-region availability.
Hosting	Docker containers on ECS Fargate or Render	No Kubernetes needed at this scale; Docker Compose for local dev keeps it portable if that changes.

Phased rollout

PHASE	SCOPE
Phase 1 – MVP ~6–8 weeks	Inbound qualification + scoring, manual + CSV intake, dashboard list/filter/export, qualified-lead email alert. Hindi + English. WhatsApp via a BSP reseller for fast pilot onboarding.
Phase 2 ~4–6 weeks	Scheduled follow-up sequences and campaigns, template management UI, remaining Indian languages (tenant-configurable), CSV import polish.
Phase 3 post-launch	Self-serve tenant onboarding (embedded signup), advanced analytics/attribution, direct Meta Cloud API migration for tenants past the volume where a BSP's margin matters.

Open questions for the client conversation

- Expected message volume and number of tenants at launch — this decides whether a BSP reseller or direct Cloud API is the right starting point.
- Which Indian languages are must-have at MVP versus acceptable to defer to Phase 2.
- Any existing LLM vendor agreement CJK wants this to standardize on.
- Whether the first cohort of clients is small enough that manual (CJK-assisted) tenant onboarding is acceptable, or whether self-serve onboarding needs to move up from Phase 3.
- Data residency or retention terms already promised to any specific client (e.g. GoodFunds) that should become the default rather than the exception.